



DIPLOMADO

Desarrollo de sistemas con tecnología Java

Módulo7

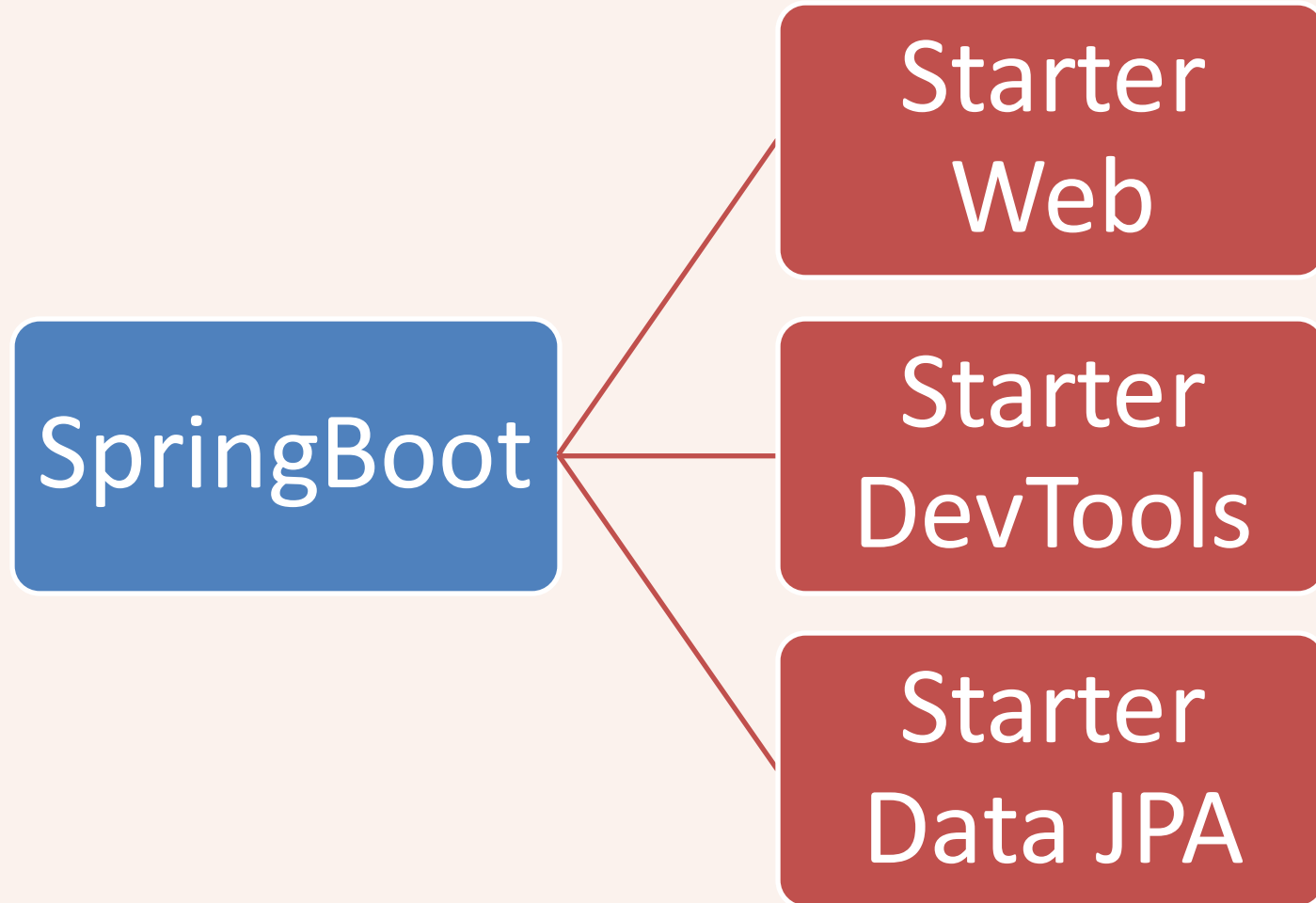
Persistencia con Spring Data

M. en C. Jesús Hernández Cabrera

jesushernandezls7@aragon.unam.mx



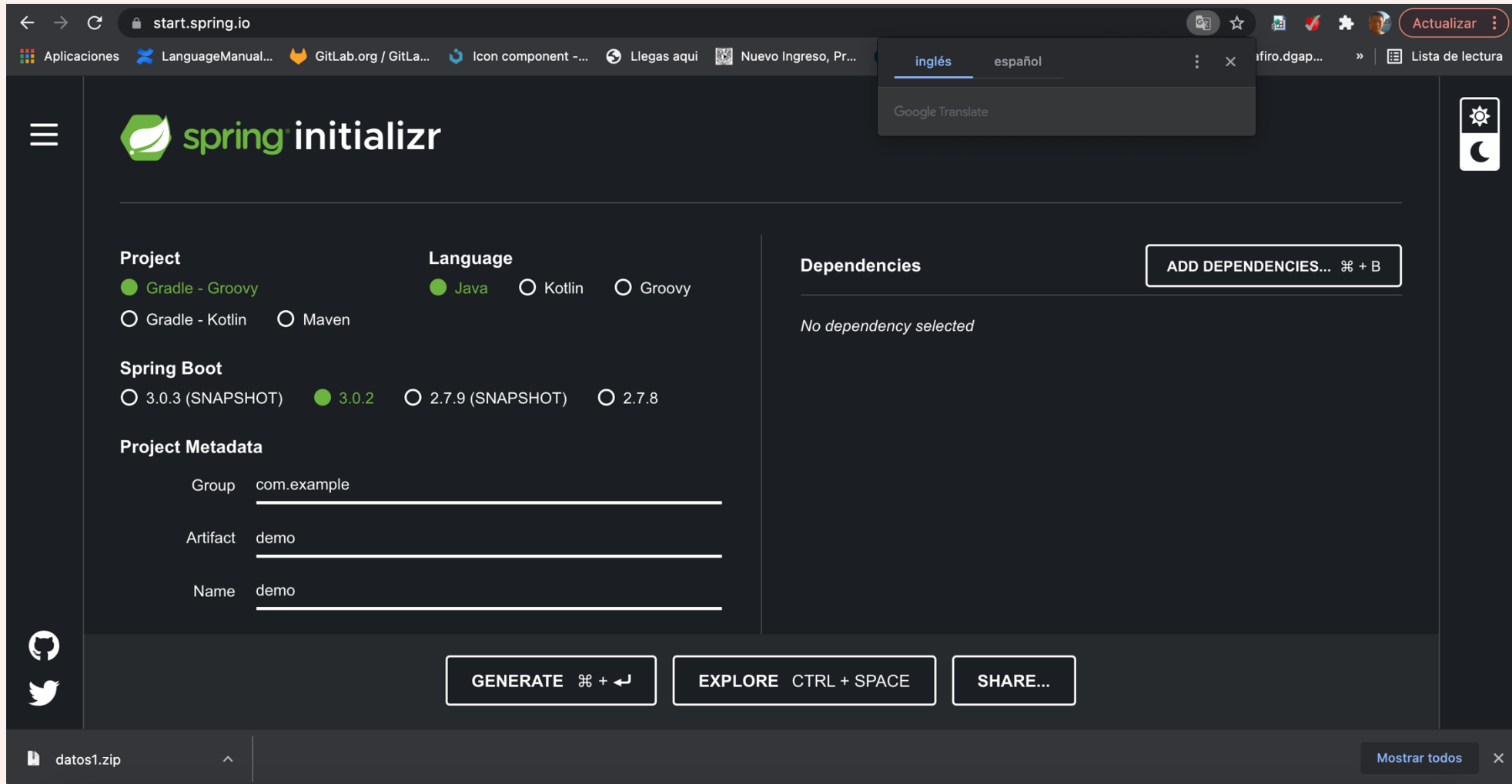
Starters Spring



Spring Data Configuración

- Spring Initializr
 - Es una aplicación web que puede generar una estructura de proyecto Spring Boot
 - <https://start.spring.io/>

Spring Data Configuración

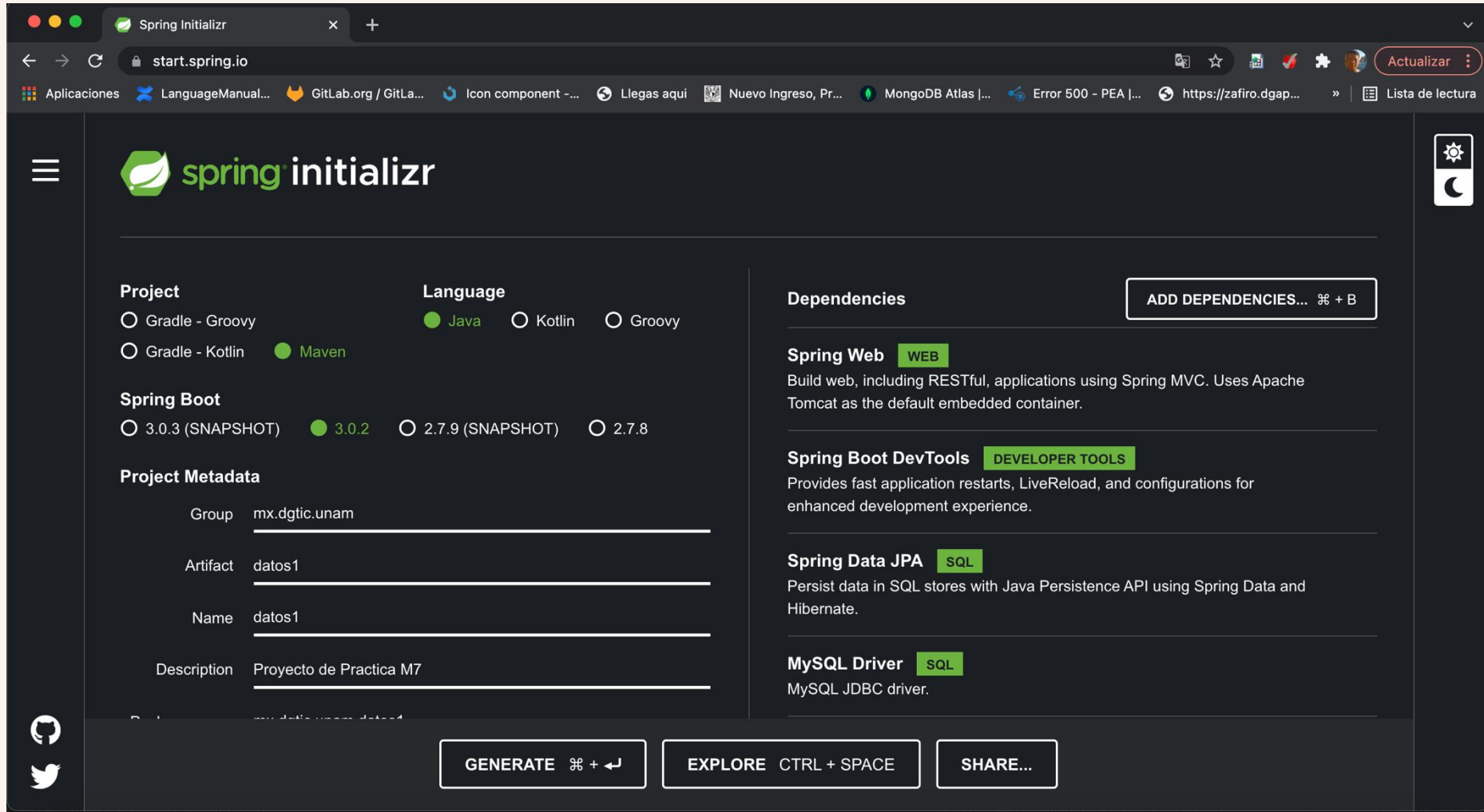


The screenshot shows the Spring Initializr web application interface. The browser address bar displays `start.spring.io`. The page features a dark theme and includes a Google Translate widget for switching between `inglés` and `español`. The main configuration area is divided into several sections:

- Project:** Includes radio buttons for `Gradle - Groovy` (selected), `Gradle - Kotlin`, and `Maven`.
- Language:** Includes radio buttons for `Java` (selected), `Kotlin`, and `Groovy`.
- Spring Boot:** Includes radio buttons for `3.0.3 (SNAPSHOT)`, `3.0.2` (selected), `2.7.9 (SNAPSHOT)`, and `2.7.8`.
- Project Metadata:** Includes input fields for `Group` (filled with `com.example`), `Artifact` (filled with `demo`), and `Name` (filled with `demo`).
- Dependencies:** Includes a button `ADD DEPENDENCIES... ⌘ + B` and the text `No dependency selected`.

At the bottom of the configuration area, there are three buttons: `GENERATE ⌘ + ↵`, `EXPLORE CTRL + SPACE`, and `SHARE...`. The bottom status bar shows a file named `datos1.zip` and a `Mostrar todos` button.

Spring Data Configuración

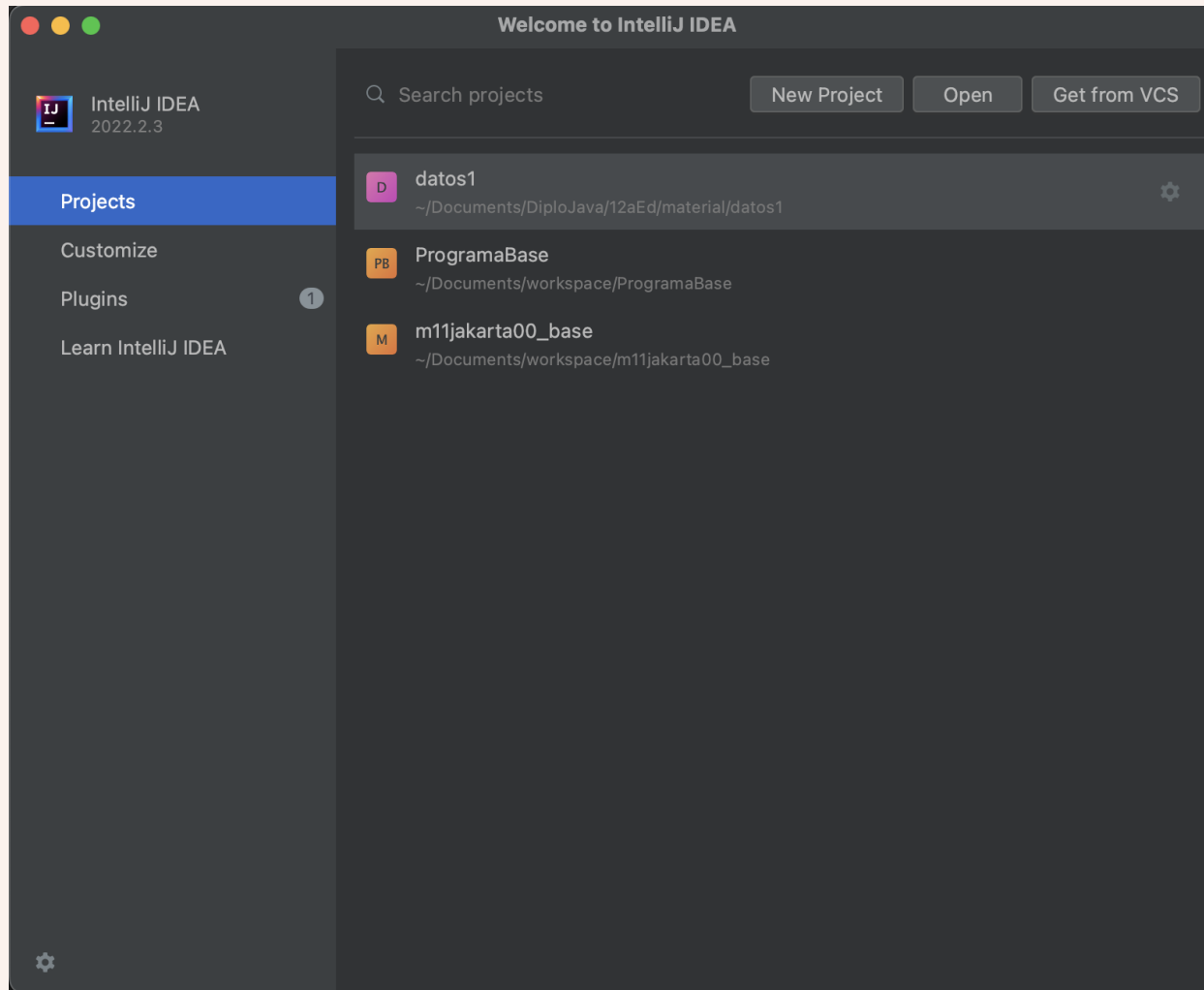


The screenshot shows the Spring Initializr web application interface. The browser address bar displays `start.spring.io`. The page features a sidebar with the Spring logo and a hamburger menu. The main content area is divided into several sections for configuring a new project:

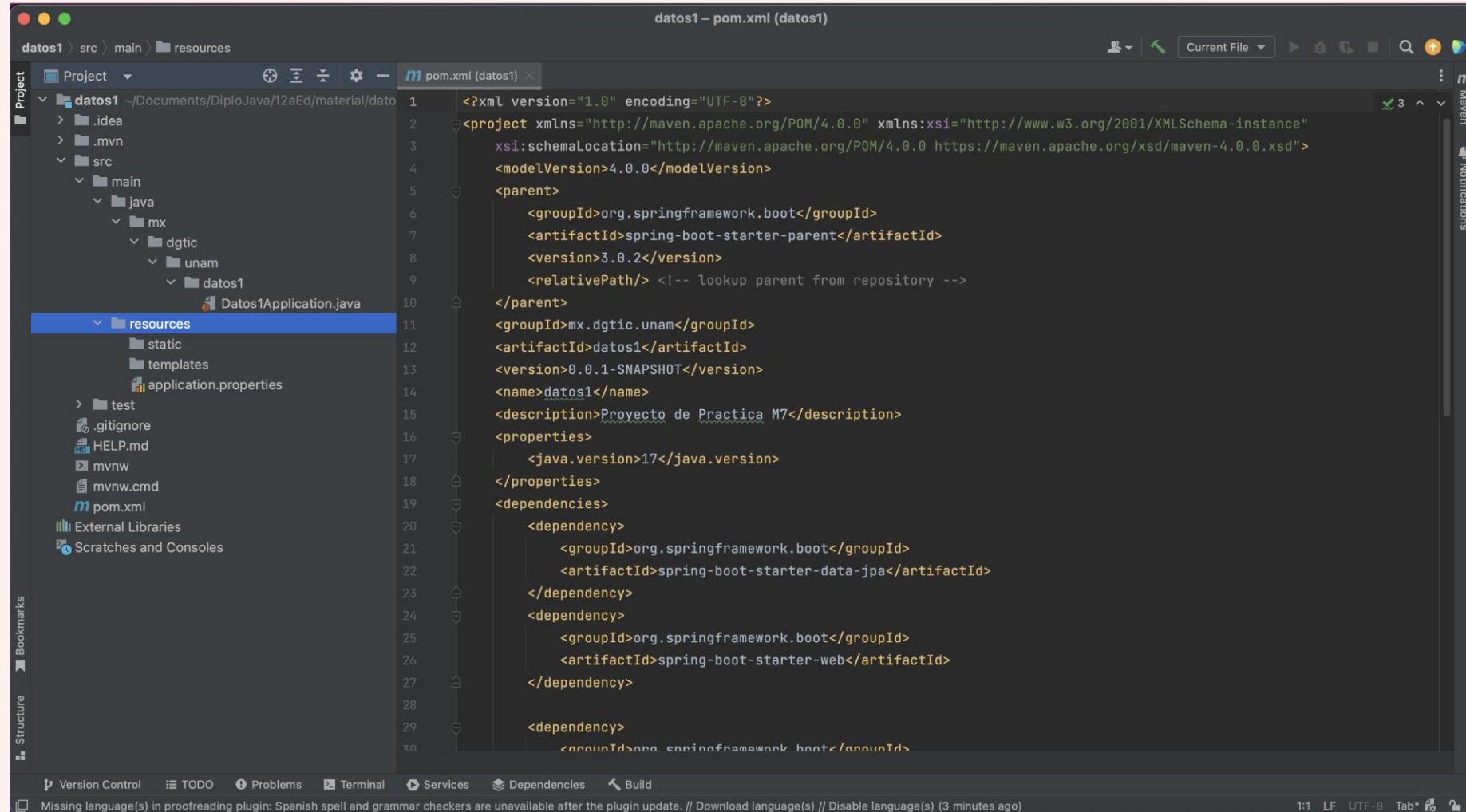
- Project:** Includes radio buttons for `Gradle - Groovy`, `Gradle - Kotlin`, and `Maven` (which is selected).
- Language:** Includes radio buttons for `Java` (selected), `Kotlin`, and `Groovy`.
- Spring Boot:** Includes radio buttons for `3.0.3 (SNAPSHOT)`, `3.0.2` (selected), `2.7.9 (SNAPSHOT)`, and `2.7.8`.
- Project Metadata:** Includes input fields for `Group` (filled with `mx.dgtic.unam`), `Artifact` (filled with `datos1`), `Name` (filled with `datos1`), and `Description` (filled with `Proyecto de Practica M7`).
- Dependencies:** A section on the right with a button `ADD DEPENDENCIES... ⌘ + B`. It lists several dependencies with checkboxes:
 - Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
 - Spring Boot DevTools** (DEVELOPER TOOLS): Provides fast application restarts, LiveReload, and configurations for enhanced development experience.
 - Spring Data JPA** (SQL): Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
 - MySQL Driver** (SQL): MySQL JDBC driver.

At the bottom of the page, there are three buttons: `GENERATE ⌘ + ↵`, `EXPLORE CTRL + SPACE`, and `SHARE...`. Social media icons for GitHub and Twitter are visible in the bottom left corner.

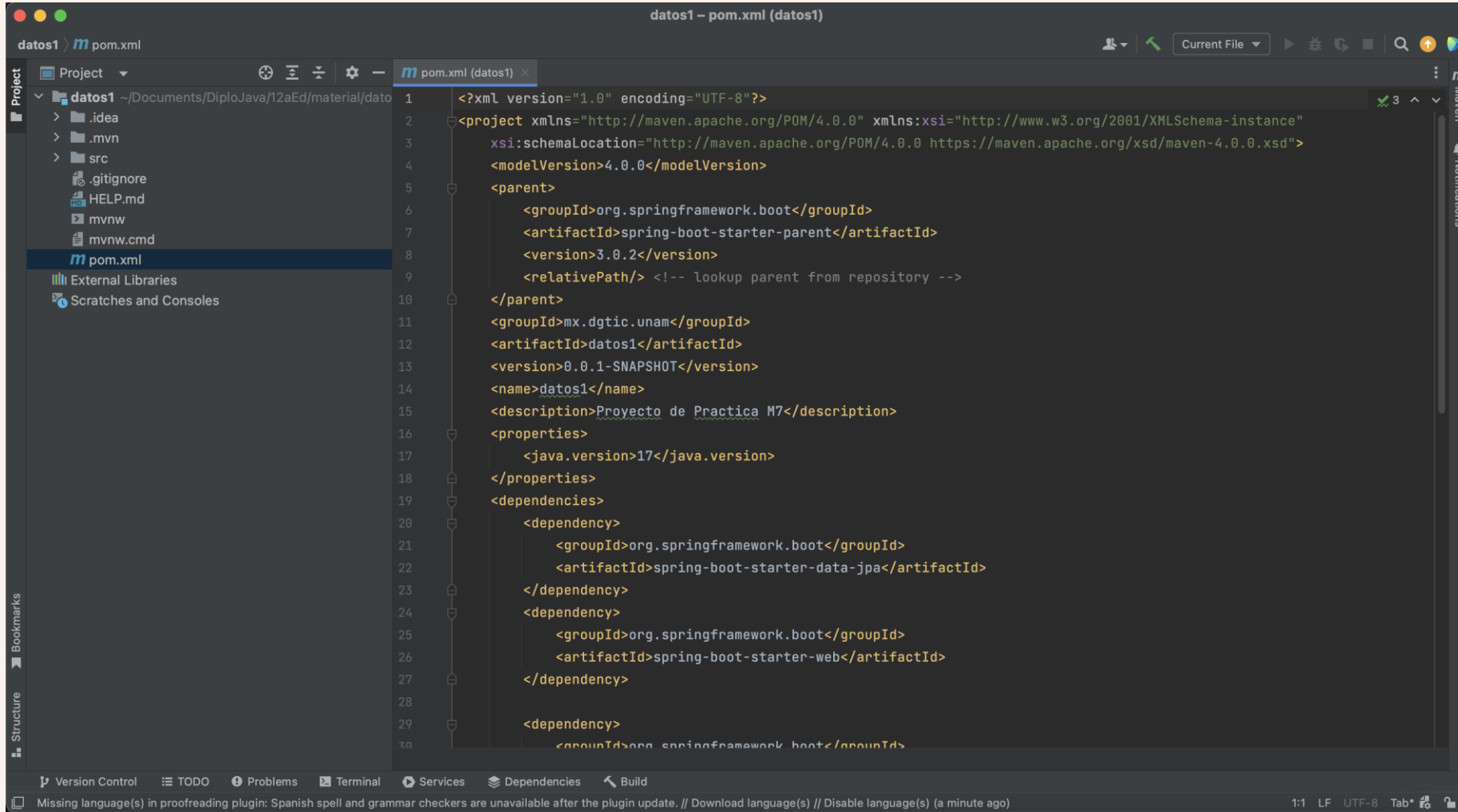
Spring Data Configuración



Spring Data Configuración

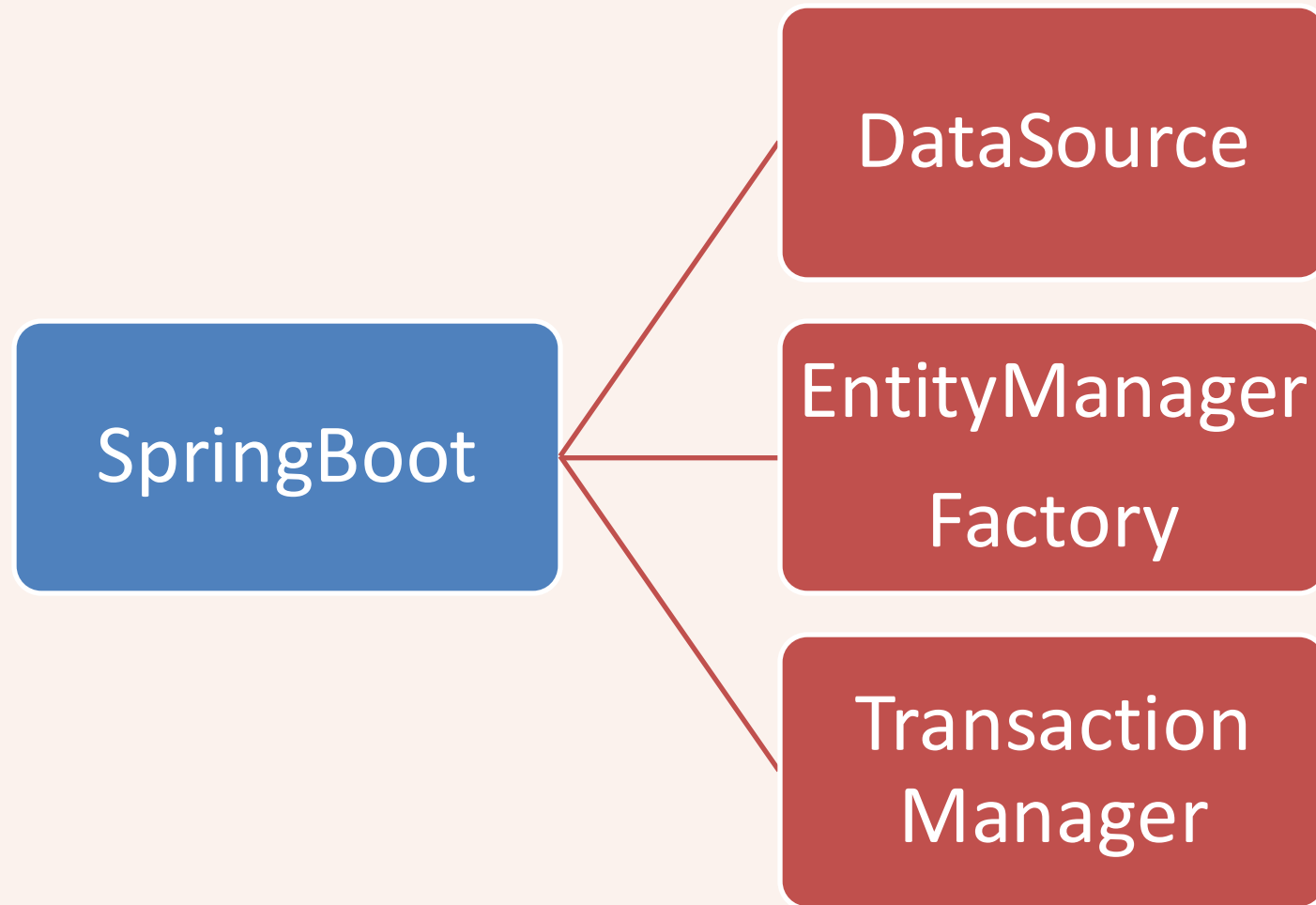


pom.xml

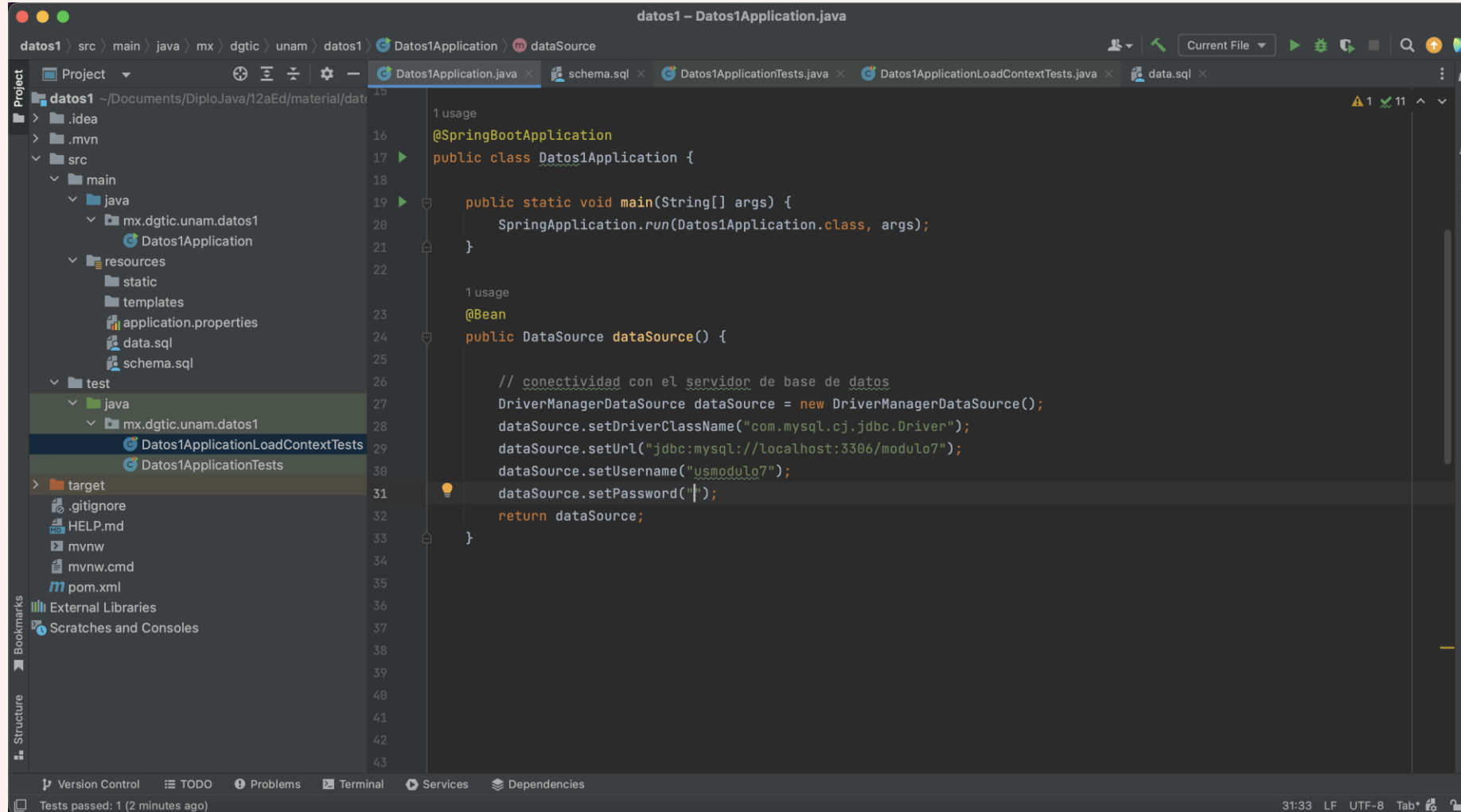


```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4   <modelVersion>4.0.0</modelVersion>
5   <parent>
6     <groupId>org.springframework.boot</groupId>
7     <artifactId>spring-boot-starter-parent</artifactId>
8     <version>3.0.2</version>
9     <relativePath/> <!-- lookup parent from repository -->
10  </parent>
11  <groupId>mx.dgtic.unam</groupId>
12  <artifactId>datos1</artifactId>
13  <version>0.0.1-SNAPSHOT</version>
14  <name>datos1</name>
15  <description>Proyecto de Practica M7</description>
16  <properties>
17    <java.version>17</java.version>
18  </properties>
19  <dependencies>
20    <dependency>
21      <groupId>org.springframework.boot</groupId>
22      <artifactId>spring-boot-starter-data-jpa</artifactId>
23    </dependency>
24    <dependency>
25      <groupId>org.springframework.boot</groupId>
26      <artifactId>spring-boot-starter-web</artifactId>
27    </dependency>
28
29    <dependency>
30      <groupId>org.springframework.boot</groupId>
```


Spring Data



Agregar DataSource

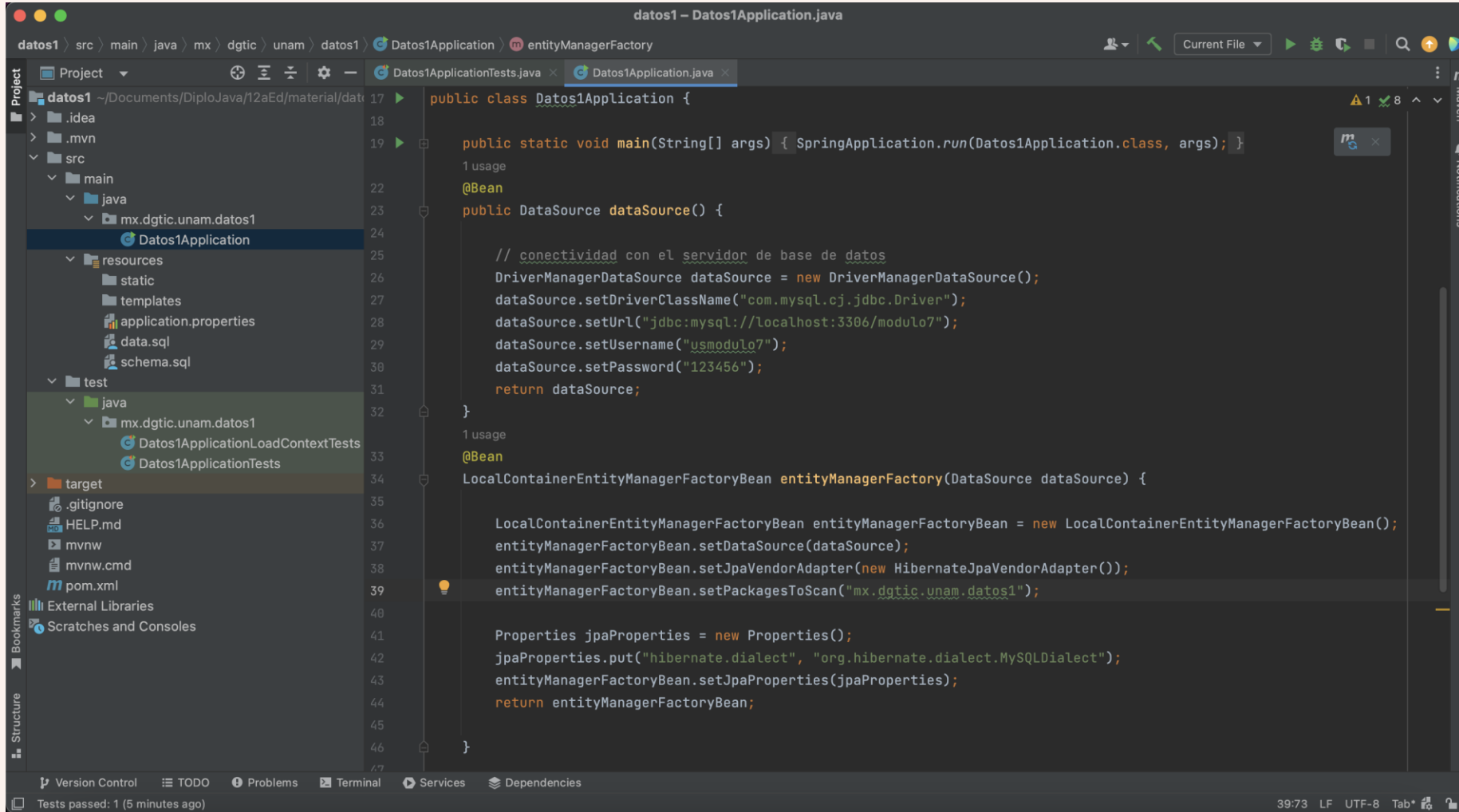


The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the project structure for 'datos1'. The 'src/main/java' directory contains 'mx.dgtic.unam.datos1', which includes 'Datos1Application'. The 'resources' directory contains 'static', 'templates', 'application.properties', 'data.sql', and 'schema.sql'. The 'test' directory contains 'mx.dgtic.unam.datos1', which includes 'Datos1ApplicationLoadContextTests' and 'Datos1ApplicationTests'.
- Code Editor:** Displays the 'Datos1Application.java' file. The code is as follows:

```
15
16
17 @SpringBootApplication
18 public class Datos1Application {
19
20     public static void main(String[] args) {
21         SpringApplication.run(Datos1Application.class, args);
22     }
23
24     @Bean
25     public DataSource dataSource() {
26
27         // conectividad con el servidor de base de datos
28         DriverManagerDataSource dataSource = new DriverManagerDataSource();
29         dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
30         dataSource.setUrl("jdbc:mysql://localhost:3306/modulo7");
31         dataSource.setUsername("usmodulo7");
32         dataSource.setPassword("");
33         return dataSource;
34     }
35
36
37
38
39
40
41
42
43
```
- Bottom Panel:** Includes tabs for 'Version Control', 'TODO', 'Problems', 'Terminal', 'Services', and 'Dependencies'. The 'Problems' tab is active, showing a warning icon and the text '1 usage'.
- Status Bar:** Shows '31:33 LF UTF-8 Tab'.

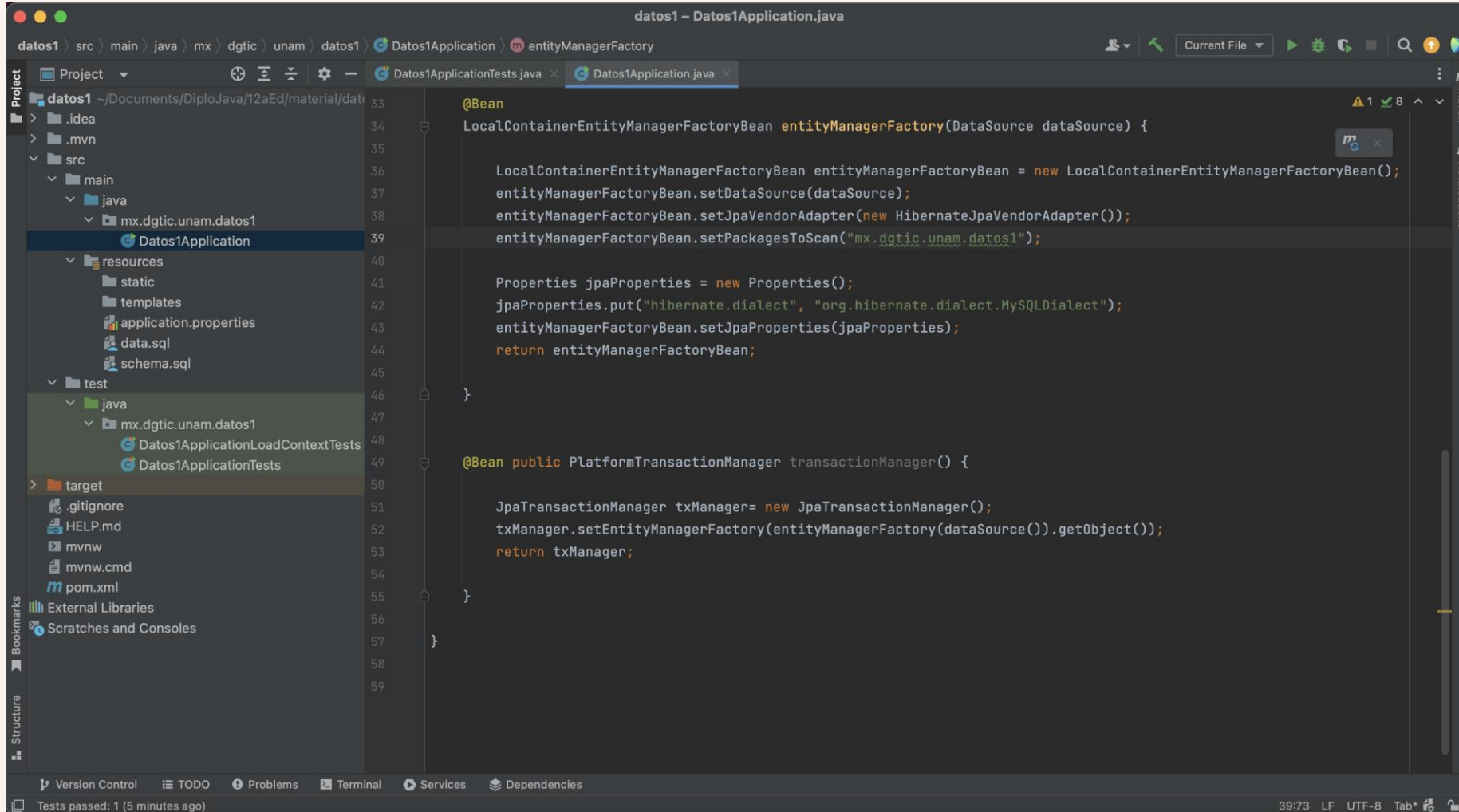
Agregar EntityManagerFactory



The screenshot shows an IDE with a project named 'datos1'. The file explorer on the left shows the project structure, including 'src/main/java/mx/dgtic/unam/datos1' and 'target'. The main editor displays the code for 'Datos1Application.java'. The code includes a main method, a DataSource method, and an EntityManagerFactory method. The EntityManagerFactory method is highlighted with a yellow background.

```
public class Datos1Application {  
    public static void main(String[] args) { SpringApplication.run(Datos1Application.class, args); }  
  
    @Bean  
    public DataSource dataSource() {  
        // conectividad con el servidor de base de datos  
        DriverManagerDataSource dataSource = new DriverManagerDataSource();  
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");  
        dataSource.setUrl("jdbc:mysql://localhost:3306/modulo7");  
        dataSource.setUsername("usuario7");  
        dataSource.setPassword("123456");  
        return dataSource;  
    }  
  
    @Bean  
    LocalContainerEntityManagerFactoryBean entityManagerFactory(DataSource dataSource) {  
        LocalContainerEntityManagerFactoryBean entityManagerFactoryBean = new LocalContainerEntityManagerFactoryBean();  
        entityManagerFactoryBean.setDataSource(dataSource);  
        entityManagerFactoryBean.setJpaVendorAdapter(new HibernateJpaVendorAdapter());  
        entityManagerFactoryBean.setPackagesToScan("mx.dgtic.unam.datos1");  
  
        Properties jpaProperties = new Properties();  
        jpaProperties.put("hibernate.dialect", "org.hibernate.dialect.MySQLDialect");  
        entityManagerFactoryBean.setJpaProperties(jpaProperties);  
        return entityManagerFactoryBean;  
    }  
}
```

Agregar TransactionManager



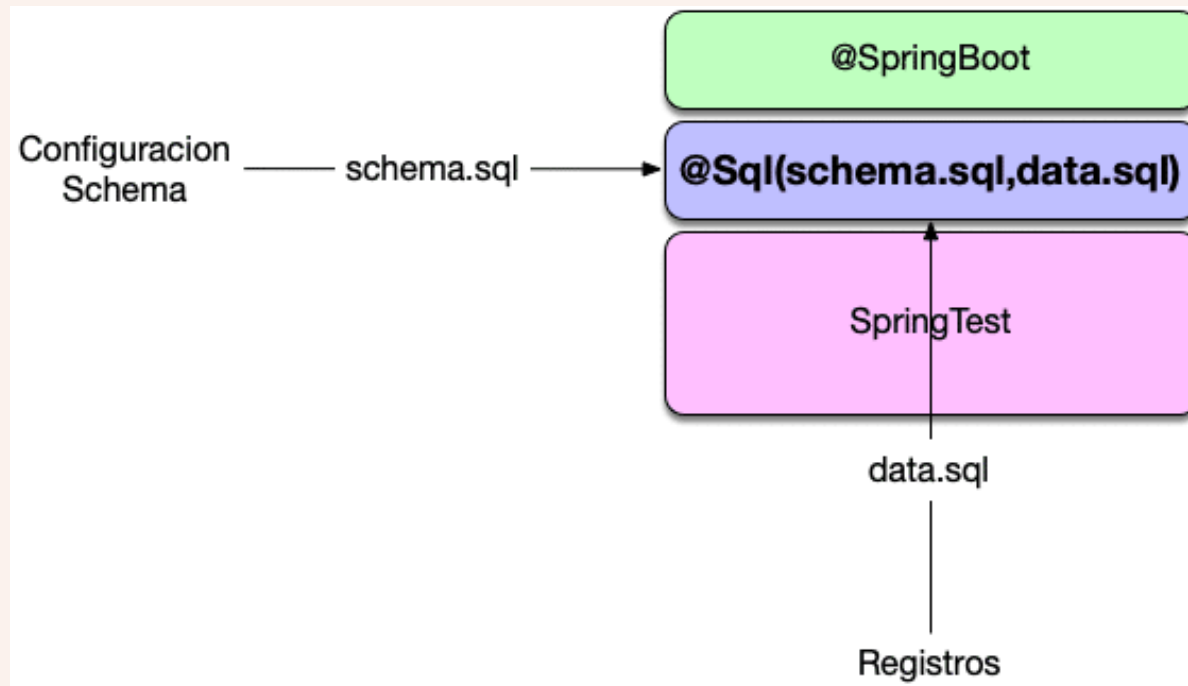
The screenshot shows an IDE with a project named 'datos1'. The file explorer on the left shows the project structure, including 'src/main/java/mx/dgtic/unam/datos1' and 'src/test/java/mx/dgtic/unam/datos1'. The main editor displays the 'Datos1Application.java' file, which contains two methods: 'entityManagerFactory' and 'transactionManager'. The 'entityManagerFactory' method is annotated with '@Bean' and returns a 'LocalContainerEntityManagerFactoryBean'. The 'transactionManager' method is also annotated with '@Bean' and returns a 'PlatformTransactionManager'.

```
datos1 - Datos1Application.java
datos1 > src > main > java > mx > dgtic > unam > datos1 > Datos1Application > entityManagerFactory
Project
  datos1 - /Documents/DiploJava/12aEd/material/datos1
  .idea
  .mvn
  src
    main
      java
        mx.dgtic.unam.datos1
          Datos1Application
            resources
              static
              templates
              application.properties
              data.sql
              schema.sql
            test
              java
                mx.dgtic.unam.datos1
                  Datos1ApplicationLoadContextTests
                  Datos1ApplicationTests
            target
              .gitignore
              HELP.md
              mvnw
              mvnw.cmd
              pom.xml
  External Libraries
  Scratches and Consoles
Structure
Version Control
TODO
Problems
Terminal
Services
Dependencies
Tests passed: 1 (5 minutes ago)
39:73 LF UTF-8 Tab*
```

```
33
34 @Bean
35 LocalContainerEntityManagerFactoryBean entityManagerFactory(DataSource dataSource) {
36
37     LocalContainerEntityManagerFactoryBean entityManagerFactoryBean = new LocalContainerEntityManagerFactoryBean();
38     entityManagerFactoryBean.setDataSource(dataSource);
39     entityManagerFactoryBean.setJpaVendorAdapter(new HibernateJpaVendorAdapter());
40     entityManagerFactoryBean.setPackagesToScan("mx.dgtic.unam.datos1");
41
42     Properties jpaProperties = new Properties();
43     jpaProperties.put("hibernate.dialect", "org.hibernate.dialect.MySQLDialect");
44     entityManagerFactoryBean.setJpaProperties(jpaProperties);
45     return entityManagerFactoryBean;
46 }
47
48 @Bean public PlatformTransactionManager transactionManager() {
49
50     JpaTransactionManager txManager= new JpaTransactionManager();
51     txManager.setEntityManagerFactory(entityManagerFactory(dataSource()).getObject());
52     return txManager;
53 }
54
55
56
57
58
59
```

@Sql

- A nivel de pruebas unitarias se dispone de la anotación **@Sql** que permite definir como se puede cargar de forma automática información en la base de datos.



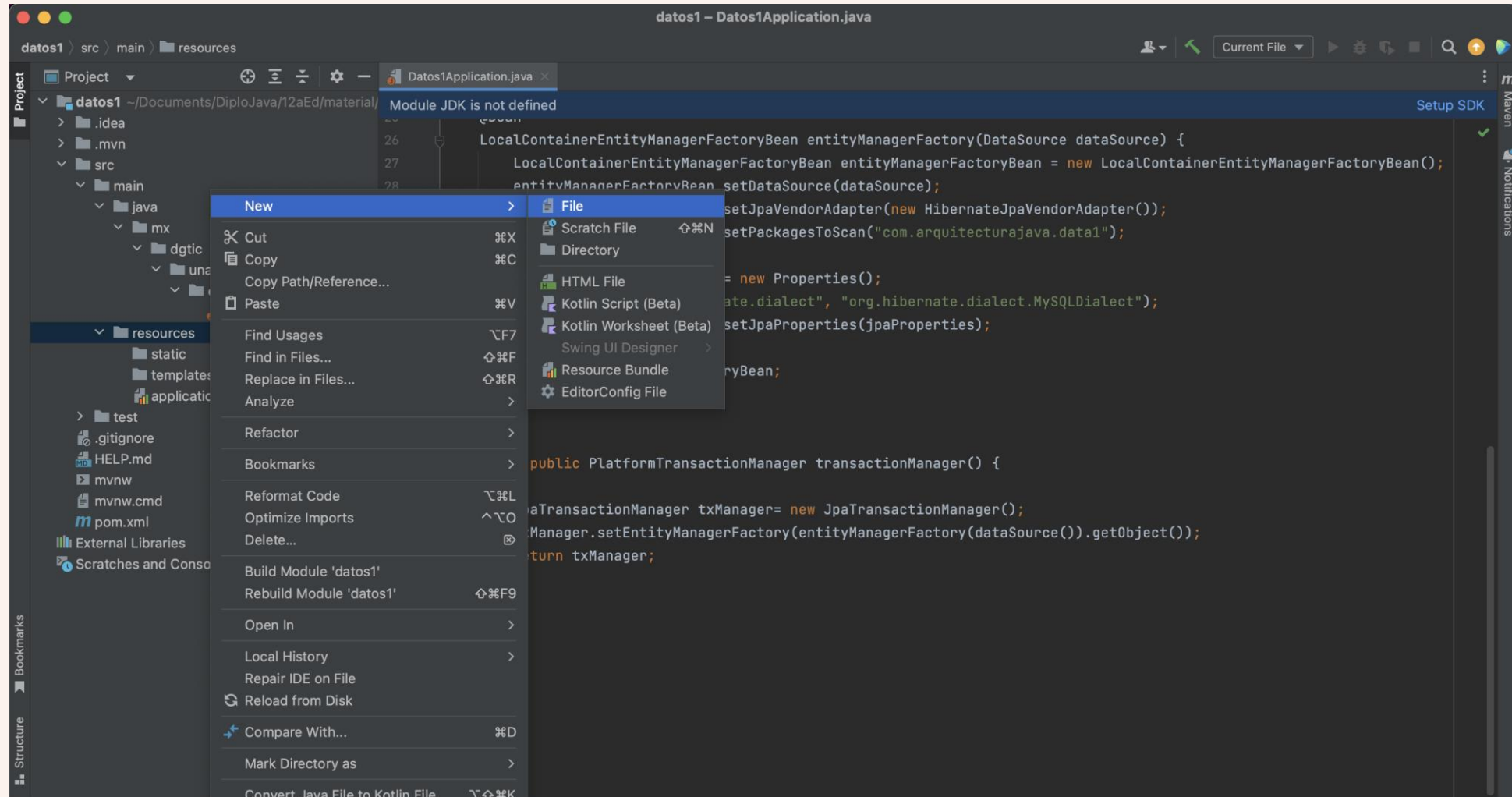
@Sql

```
@Sql(schema.sql, data.sql)
```

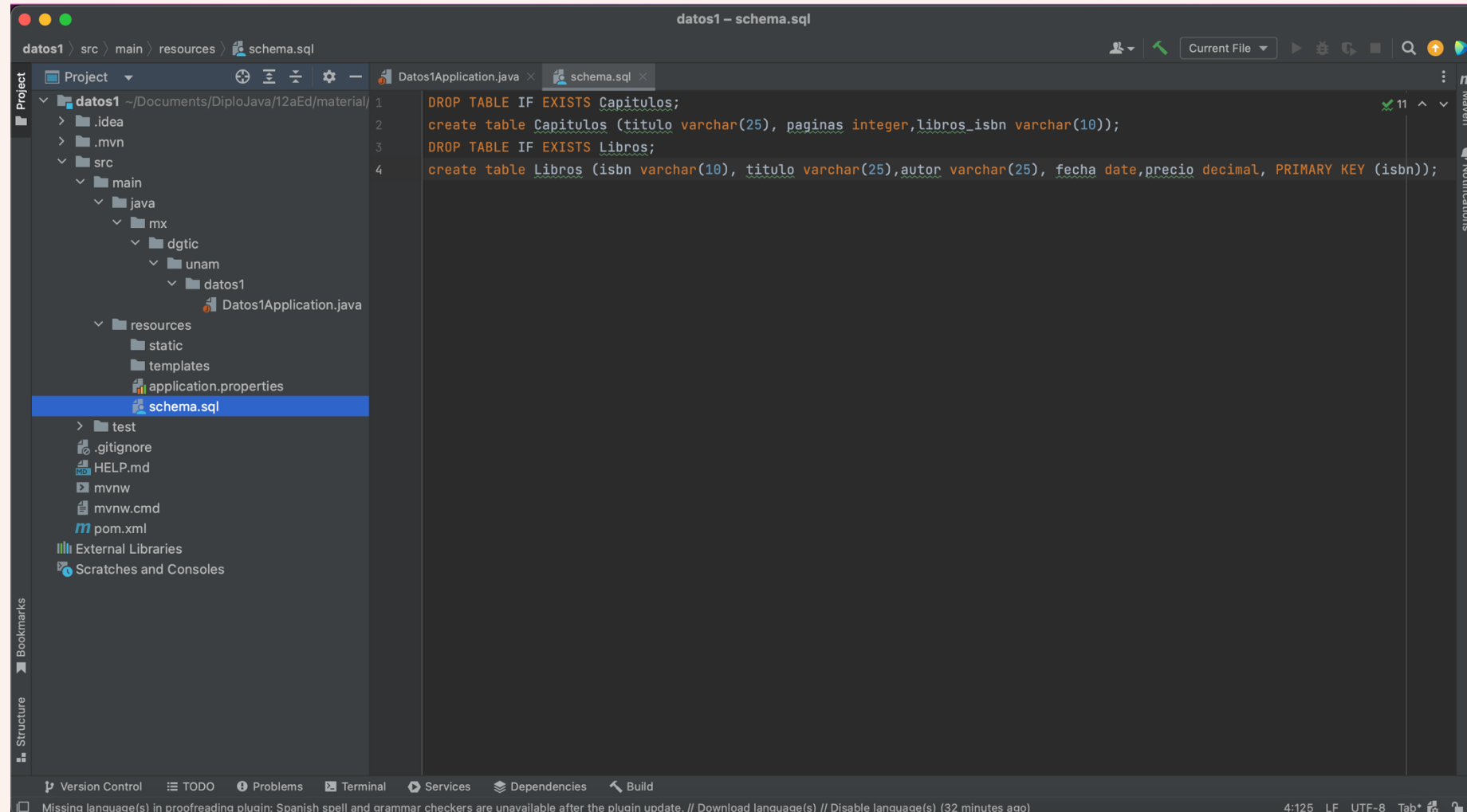
Create

insert

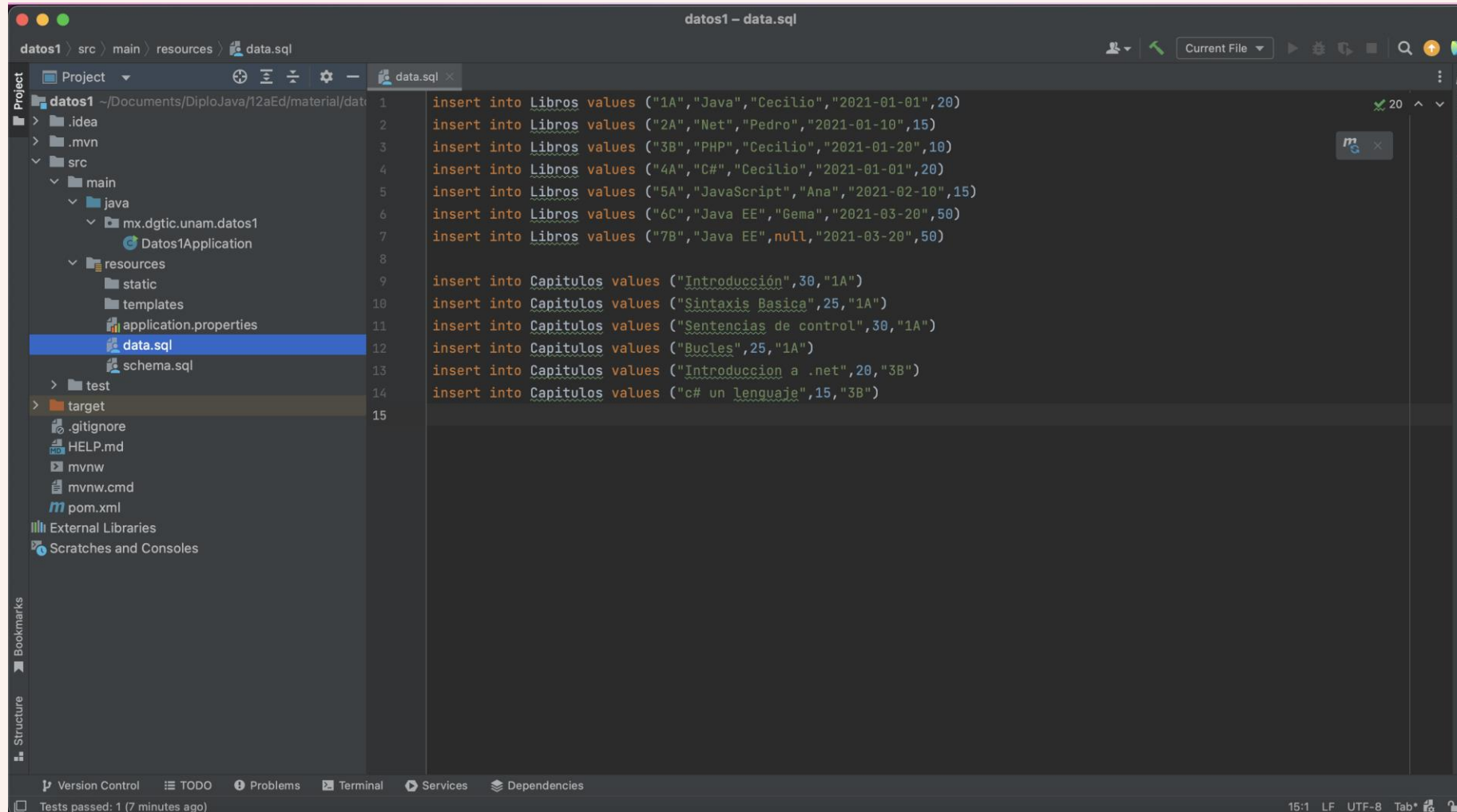
Agregar archivos sql



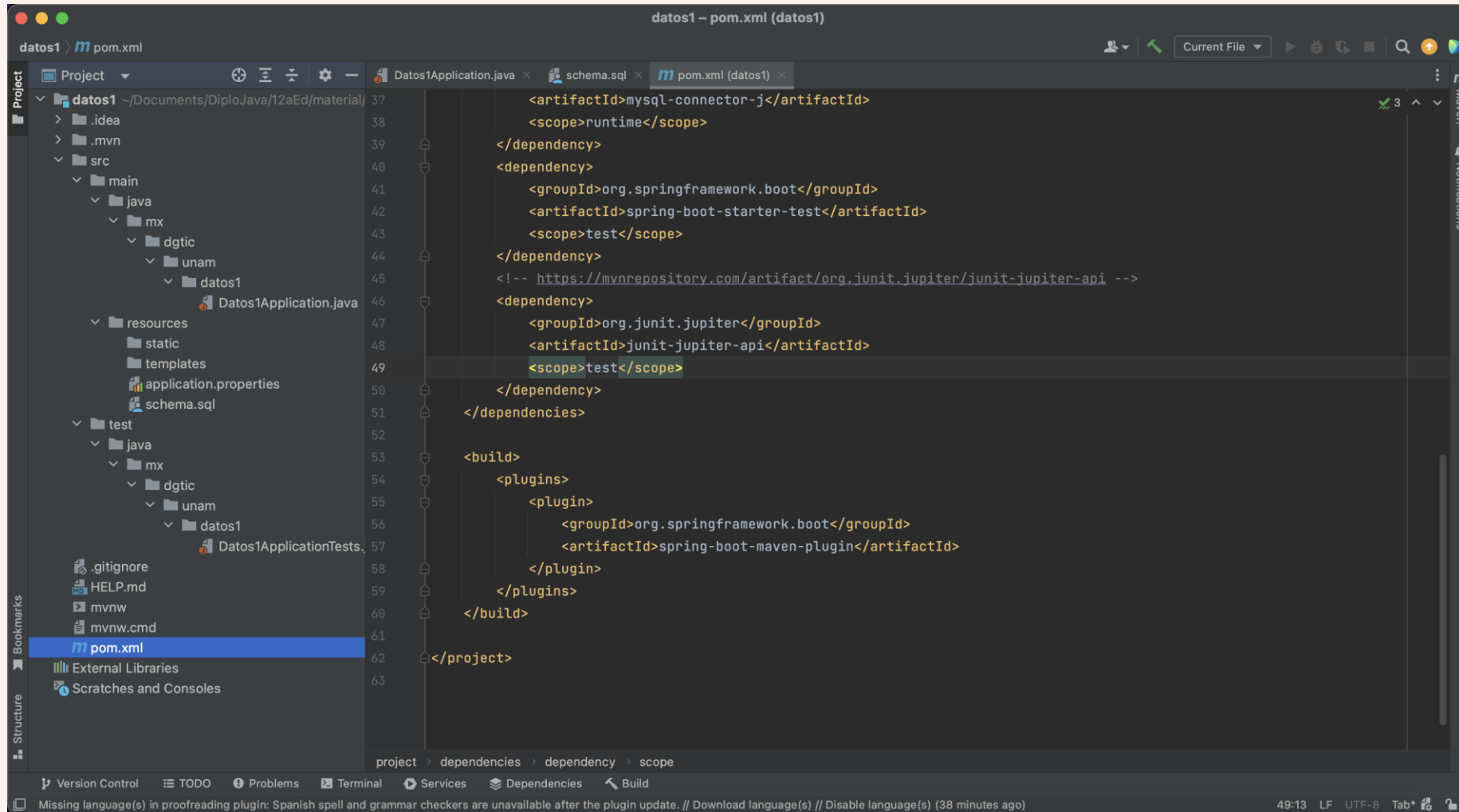
Agregar archivo schema.sql



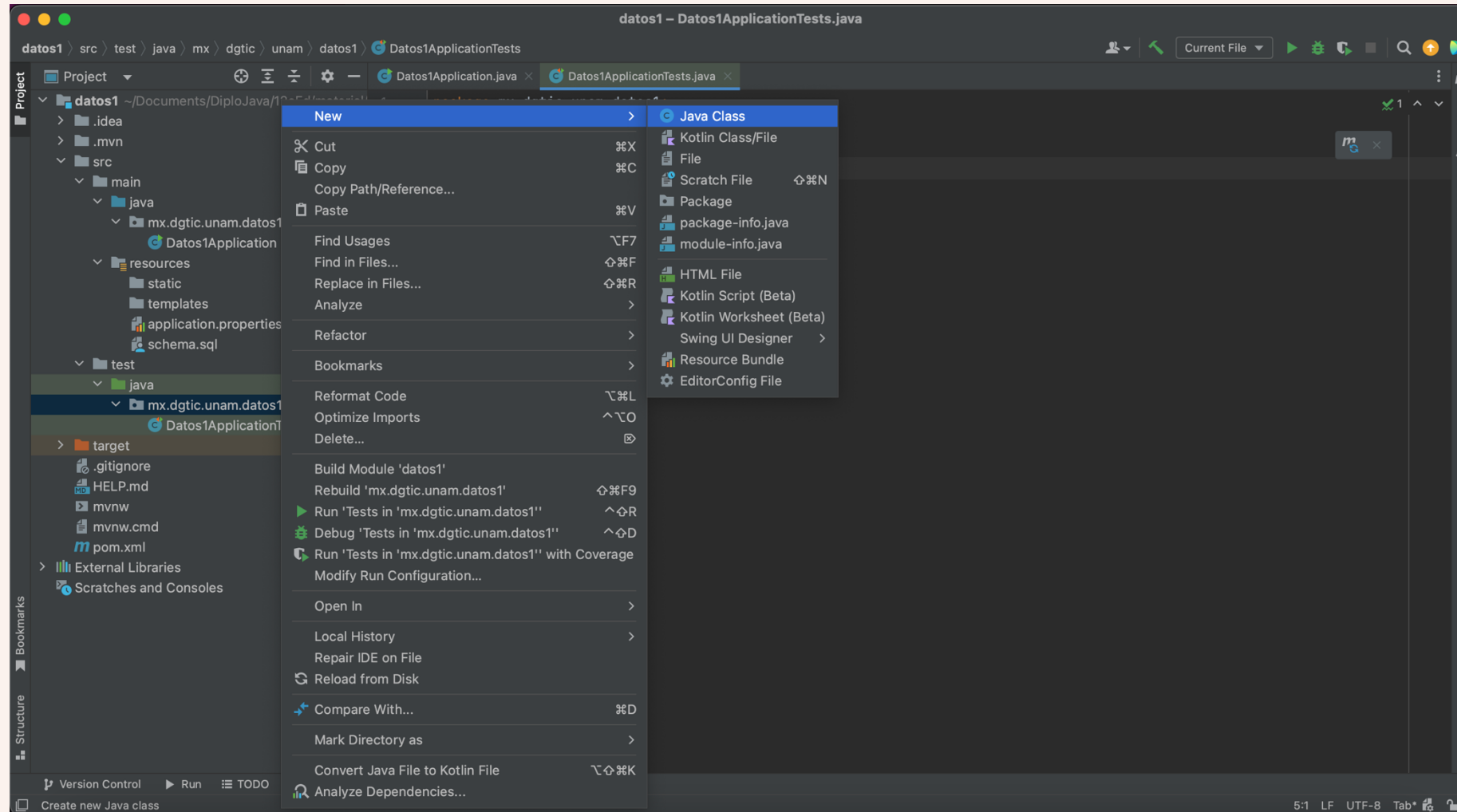
Agregar archivo data.sql



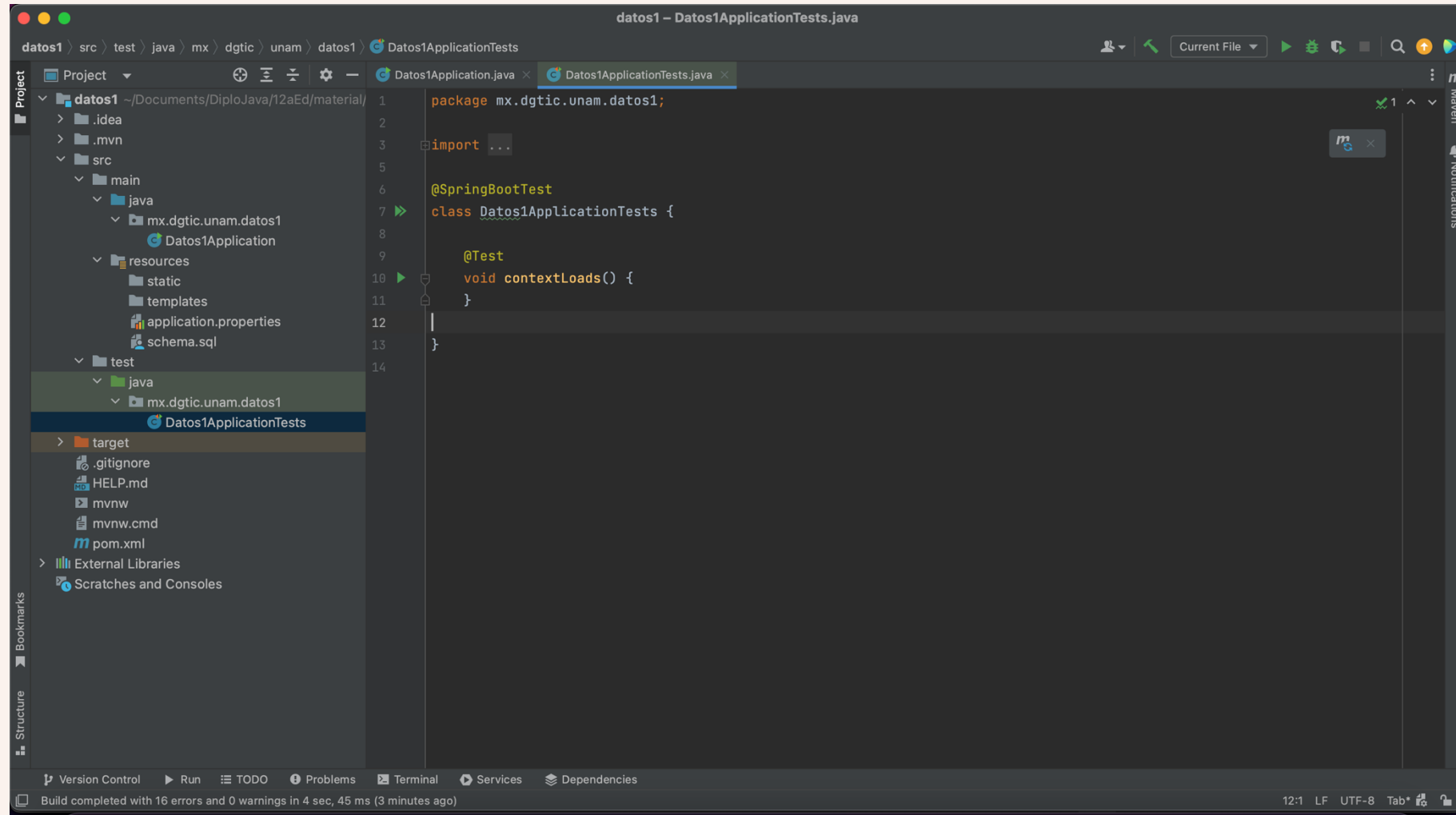
Agregar dependencia JUnit a pom.xml



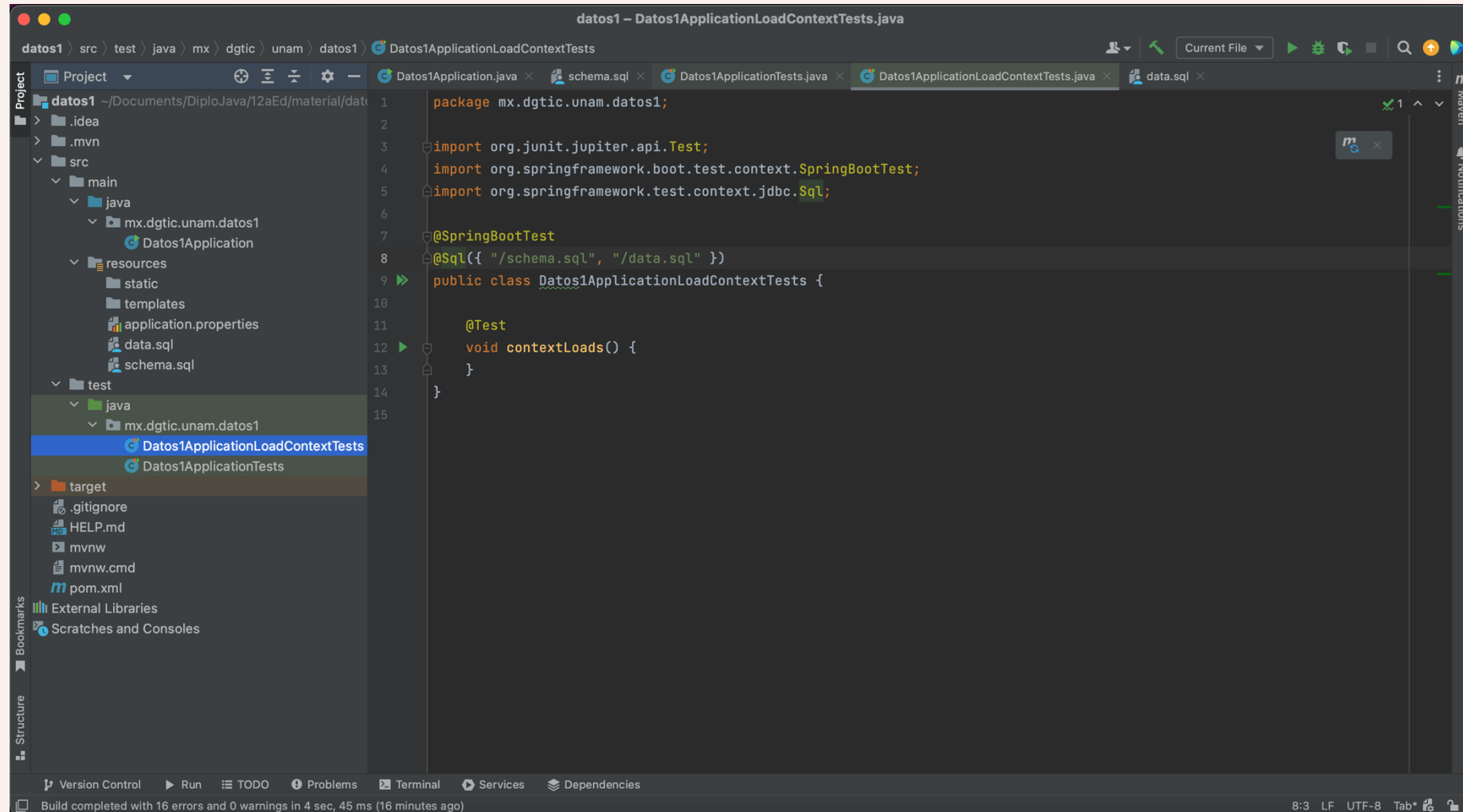
Agregar JUnitTest



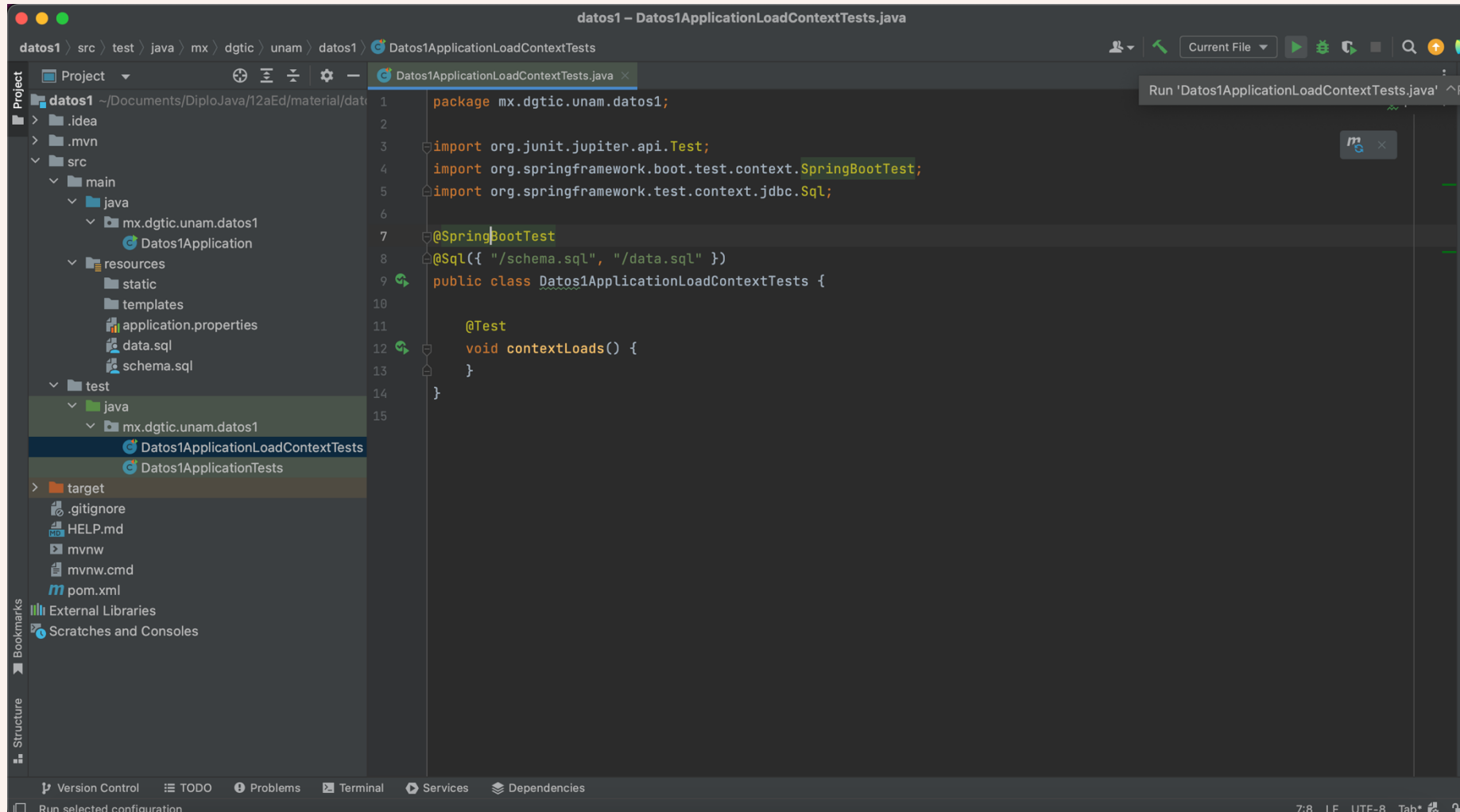
Agregar JUnitTest



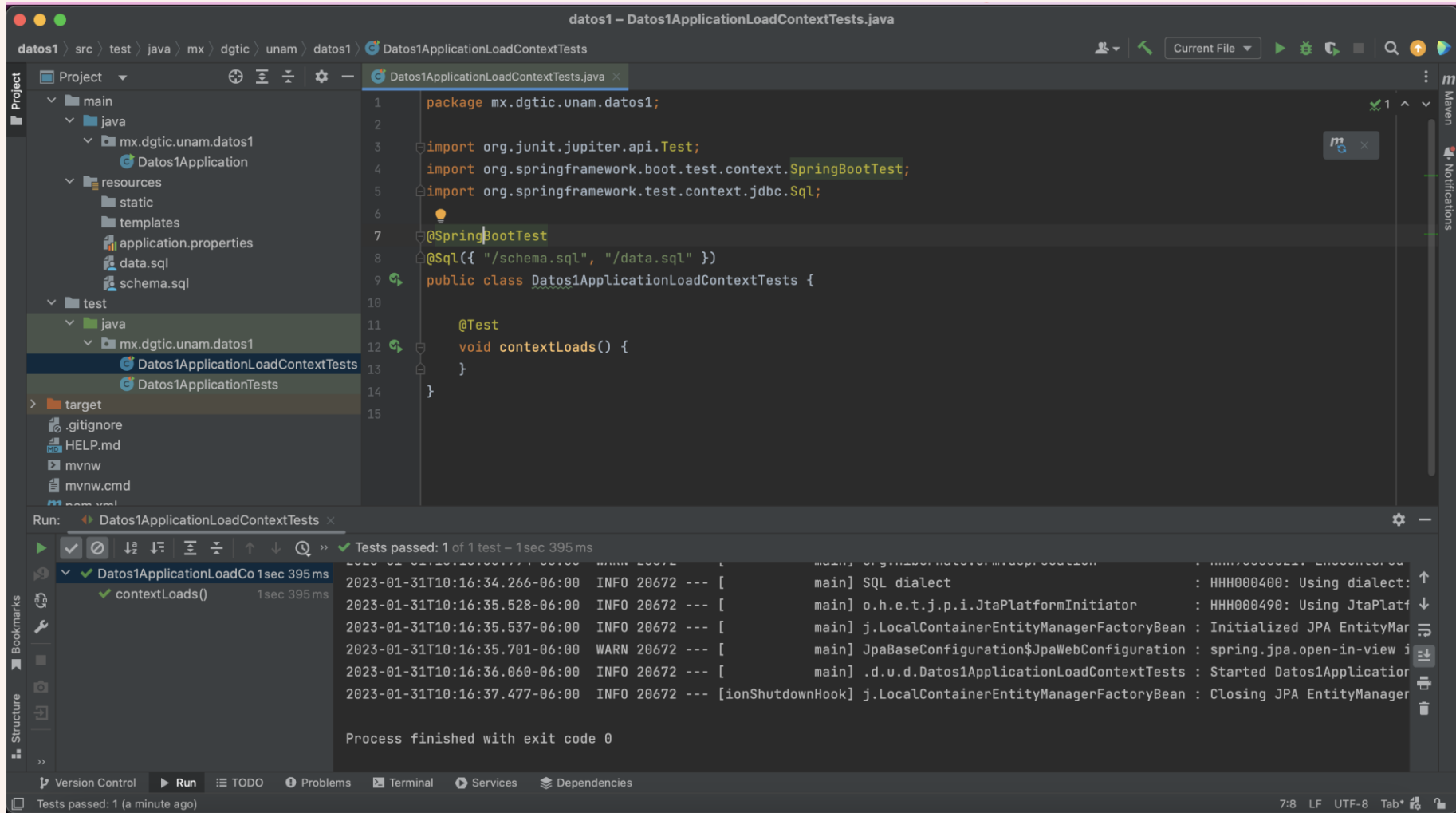
Definir datos para prueba



Ejecutar Test



Ejecutar Test



The screenshot shows an IDE window titled "datos1 - Datos1ApplicationLoadContextTests.java". The project structure on the left includes a "test" directory with a "java" subdirectory containing "Datos1ApplicationLoadContextTests". The main editor displays the following code:

```
1 package mx.dgtic.unam.datos1;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5 import org.springframework.test.context.jdbc.Sql;
6
7 @SpringBootTest
8 @Sql({ "/schema.sql", "/data.sql" })
9 public class Datos1ApplicationLoadContextTests {
10
11     @Test
12     void contextLoads() {
13     }
14 }
15
```

The "Run" tab at the bottom shows the execution results for "Datos1ApplicationLoadContextTests". The test "contextLoads()" passed successfully in 1sec 395ms. The output log shows the following messages:

```
2023-01-31T10:16:34.266-06:00 INFO 20672 --- [main] SQL dialect : HHH000400: Using dialect:
2023-01-31T10:16:35.528-06:00 INFO 20672 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000490: Using JtaPlatform:
2023-01-31T10:16:35.537-06:00 INFO 20672 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2023-01-31T10:16:35.701-06:00 WARN 20672 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. This can lead to stale data being left behind in tables if your application does not take an initiative to flush the database immediately after each update. This behavior is deprecated in all versions of Spring. Consider setting the 'spring.jpa.open-in-view' property to false.
2023-01-31T10:16:36.060-06:00 INFO 20672 --- [main] .d.u.d.Datos1ApplicationLoadContextTests : Started Datos1Application in 1.395 seconds
2023-01-31T10:16:37.477-06:00 INFO 20672 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
```

The process finished with exit code 0.

Contacto

M. en C. Jesús Hernández Cabrera
Profesor de carrera

jesushc@unam.mx

Redes sociales:



www.linkedin.com/in/hcjesus